Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Физико-механический институт
Высшая школа прикладной математики и вычислительной физики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

по курсу «Введение в технологии суперкомпьютерных вычислений»
на тему: «Изучение базовых конструкций технологии OpenMP на примере параллелизации программы вычисления определенного интеграла»

Работу выполнила:
Студент гр. 5030301/40102
Набережных Виктория Александровна

Преподаватели:
доцент ВШПМиВФ, к.т.н. Абрамов Алексей Геннадьевич

Санкт-Петербург
2026

Оглавление

1. Цель работы и постановка задачи 3
2. Информация об аппаратном и программном обеспечении 4
3. Исследование оптимизации компилятора и автоматического распараллеливания 4
4. Формулировка параллельного алгоритма для соответствующей операции 5
5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP 6
6. Результаты проверки правильности выполнения программы 8
7. Результаты исследования эффективности распараллеливания программы 9
8. Результаты исследования эффективности способов распределения итераций цикла . 10
Выводы 11
Приложение (код программы) 12

1. Цель работы и постановка задачи

Целью данной работы является изучение функций и возможностей базовых конструкций технологии OpenMP применительно к распараллеливанию программы вычисления определенного интеграла.

Для достижения поставленной цели необходимо разработать программу, вычисляющую определенный интеграл на конечном промежутке по методу средних прямоугольников и выполнить ее распараллеливание средствами OpenMP.

Математическая постановка задачи: вычислить определенный интеграл I заданной непрерывной функции f(x) на отрезке [0, 1]:

$$I = \int_0^1 \frac{ln(x)}{1+x} dx \, , dx = -\frac{\pi^2}{12}$$

Для численного интегрирования используется метод средних прямоугольников. Суть метода заключается в аппроксимации площади под кривой суммой площадей элементарных прямоугольников. Исходный отрезок интегрирования [a, b] разбивается на N равных элементарных подотрезков с шагом:

$$h = \frac{b\ -\ a}{N}$$

Интегральная сумма вычисляется путем взятия значений интегрируемой функции в средних точках каждого элементарного подотрезка. Приближенное значение интеграла I определяется по формуле:

$$I \approx h \sum_{i=0}^{N-1} f(x_i)$$

где $x_i = a + (i + 0{,}5)h$ – координата середины i-го подотрезка.

В программе, написанной на языке Fortran, расчет ведется для i от 1 до N. Итоговая вычислительная формула:

$$I \approx h \sum_{i=1}^{N} f(a + (i - 0{,}5)h)$$

Чтобы вычисления занимали заметное процессорное время (порядка 0.1 - 1.0 секунды) и накладные расходы на создание потоков не перекрывали полезную работу, число разбиений выбирается достаточно большим: N = 10^8.

Для анализа производительности введены следующие метрики:

1. *Время выполнения Tp* – измеряется с помощью функции реального(астрономического) времени omp_get_wtime(), которая возвращает время в секундах (тип real(8))

2. *Ускорение Sp* - на p потоках по отношению к последовательной версии T1:

$$Sp = \frac{T1}{Tp}$$

3. *Эффективность распараллеливания Ep:*

$$Ep = \frac{Sp}{p} = \frac{T1}{p \cdot Tp}$$

В идеальном случае $Sp \approx p$, а $Ep \approx 1.0$ (100%).

2. Информация об аппаратном и программном обеспечении

Для проведения вычислительных экспериментов и замера времени работы программы использовался ноутбук на базе операционной системы macOS. Написание и редактирование исходного кода выполнялось в текстовом редакторе Geany, а автоматизация прогонов - с помощью командных скриптов Bash.

Аппаратная конфигурация компьютера:

Процессор: Apple M1 (архитектура ARM64, 8 ядер: 4 производительных и 4 энергоэффективных)

Оперативная память: 16 ГБ единой памяти (Unified Memory)
Максимальное число потоков: в среде OpenMP процессор определяется как 8-поточный

Использованное программное обеспечение:

Компилятор: gfortran (GNU Fortran, установлен через пакетный менеджер Homebrew)

Ключи компиляции (Release): флаг -O3 для включения максимальной оптимизации кода компилятором и флаг -fopenmp для поддержки многопоточных директив

Инструменты анализа правок: среда Jupyter Notebook, язык Python 3 с библиотеками pandas (обработка таблиц) и matplotlib (визуализация и графики)

3. Исследование оптимизации компилятора и автоматического распараллеливания

Было исследовано, как компилятор gfortran умеет самостоятельно оптимизировать код и может ли он автоматически раскидать цикл по ядрам без использования директив OpenMP.

Эксперимент проводился на последовательном алгоритме (SERIAL) при фиксированном числе разбиений N = 100 000 000. Программа была скомпилирована и запущена в двух режимах:

1. **Режим Debug** (без оптимизации, флаг -O0):

```
gfortran -O0 -fopenmp lab1_sct.f90 -o integral_debug
```

Компиляция выполняется в соответствии с последовательностью исходного кода. Время выполнения составило 0,459721 сек.

2. **Режим Release** (с максимальной оптимизацией, флаг -O3):

gfortran -O3 -fopenmp lab1_sct.f90 -o integral_fortran

Компилятор пытается перестроить код, развернуть мелкие циклы и задействовать векторные инструкции процессора для одновременной обработки нескольких чисел. Время выполнения составило 0.331244 сек.

Таким образом, можно сказать, что включение флага -O3 сократило время расчета последовательной версии с 0,46 до 0,33 секунд (около 28%). Такой прирост скорости объясняется тем, что компилятор с флагом -O3 избавился от постоянных вызовов математической функции и вставил формулу логарифма прямо внутрь цикла. Процессор стал тратить меньше времени на переходы по памяти, за счет чего и сократилось время работы программы.

Попытка заставить компилятор автоматически распараллелить код (без написания директив OpenMP) не сработала. За счет флага -O3 время уменьшилось с 0.45 до 0.33 секунды, но этот прирост скорости произошел только из-за ускорения самой математики логарифма. Понятно, что компилятор не задействовал другие ядра, потому что время параллельных методов (например, REDUCTION) составило всего 0.07 секунды. Если бы компилятор сам включил многопоточность, для SERIAL был бы получен такой же быстрый результат. Раз этого не произошло, значит, весь цикл считался строго на одном ядре. Это доказывает необходимость использования OpenMP.

4. Формулировка параллельного алгоритма для соответствующей операции

В работе математическая функция во всех шести вариантах $f(x) = \frac{\ln(x)}{1+x}$. Расчет определенного интеграла производился на отрезке [0,1] по формуле средних прямоугольников с фиксированным числом разбиений $N = 10^8$. Считаем методом средних прямоугольников. Весь отрезок от 0 до 1 делится на множество мелких частей - прямоугольников, у каждого из которых ширина равна шагу h = 1/N.

Параллельный алгоритм распределяет вычисления между p потоками OpenMP и состоит из этапов:

Распределение итераций (витков цикла):

Общее число прямоугольников N делится между p работающими потоками. В автоматических методах компилятор сам нарезает цикл на непрерывные блоки. В ручном методе (SPMD) потоки берут итерации циклически с шагом p (поток с номером tid вычисляет прямоугольники tid + 1, tid + 1 + p, tid + 1 + 2p и т.д.)

Локальные вычисления:

Каждый поток работает независимо в рамках своих итераций. Он находит координату xi в середине текущего подотрезка по формуле: xi=a+(i-0,5)h. Вычитание 0,5 сдвигает точку вправо и позволяет избежать неопределённости логарифма от нуля на левой границе x=0. Далее поток вычисляет высоту прямоугольника f(xi) = ln(xi)/(1+xi) и накапливает сумму их площадей в переменную.

Синхронизация результатов:

После завершения цикла локальные суммы всех потоков должны объединиться в общую переменную. Чтобы избежать гонки данных, используются разные механизмы синхронизации OpenMP:

reduction: автоматическое слияние приватных переменных силами компилятора
critical: организация программной очереди для поочерёдного доступа к общей переменной
atomic: неделимое сложение на уровне одной быстрой инструкции процессора
FALSE_SHARING: запись результатов в общий массив по индексу потока

Получение итогового значения:

После слияния всех частичных сумм в единый глобальный накопитель главный поток один раз умножает результат на шаг h. Полученное значение является финальным ответом.

№	Вариант	Суть подхода
1	Последовательный (SERIAL)	Расчет интеграла на одном потоке (ядре) без использования технологий OpenMP
2	SPMD-вариант (SPMD)	Запускается только параллельная область !$omp parallel. Каждый поток узнает свой номер tid и общее число потоков nthr, вручную делит диапазон итераций и считает свою локальную сумму.
3	Parallel Do + Reduction (REDUCTION)	Цикл делится между потоками автоматически директивой parallel do, а опция reduction(+:sum) создает приватную копию суммы в каждом потоке и складывает их в конце.
4	Parallel Do + Atomic (ATOMIC)	Цикл делится между потоками автоматически директивой parallel do, каждый поток накапливает локальную частичную сумму, а затем все потоки прибавляют значение к одной общей переменной через аппаратную атомарную инструкцию (atomic).
5	Parallel Do + Critical (CRITICAL)	То же самое автоматическое деление цикла, но для сложения частичных сумм используется создание критической секции с помощью директивы critical
6	Ложное разделение (FALSE_SHARING)	Чтобы избежать гонки без reduction, создается общий массив double sum_arr[num_threads]. Поток с номером tid пишет только в свою ячейку sum_arr[tid].

5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP

Полный код программы представлен в приложении. Рассмотрим коды расчетных функций на языке Fortran, реализующие последовательный алгоритм интегрирования и 5 параллельных модификаций OpenMP

Во всех параллельных версиях соблюдены следующие принципы:

Конструкции *private* используются для изоляции переменных внутри каждого потока (счетчики циклов, координаты x), что исключает наложение данных.

Конструкции *shared* открывают потокам доступ к общим неизменяемым параметрам (границы отрезка, шаг интегрирования)

Для объединения промежуточных результатов в итоговую сумму применены различные средства синхронизации (reduction, atomic, critical), предотвращающие гонку данных (Data Race).

```
function integrate_sequential(a, b, n) result(res)
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, s, x
    integer(8) :: i

    h = (b - a) / real(n, 8)
    s = 0.0d0
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        s = s + log(x) / (1.0d0 + x)
    end do
    res = s * h
end function integrate_sequential
```

1.Последовательный (SERIAL)

```
function integrate_spmd(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, total_sum, local_sum, x
    integer(8) :: i
    integer :: tid, nt

    h = (b - a) / real(n, 8)
    total_sum = 0.0d0

    !$omp parallel private(tid, nt, i, x, local_sum) shared(a, h, n, total_sum)

    tid = omp_get_thread_num()
    nt = omp_get_num_threads()
    local_sum = 0.0d0

    do i = tid + 1, n, nt
        x = a + (real(i, 8) - 0.5d0) * h
        local_sum = local_sum + log(x) / (1.0d0 + x)
    end do

    !$omp atomic
    total_sum = total_sum + local_sum
    !$omp end parallel

    res = total_sum * h
end function integrate_spmd
```

2.SPMD-вариант

В **последовательном** варианте реализуется обычный цикл, выполняющийся строго последовательно на одном процессоре.

В варианте с ручным распределением итераций **(SPMD)** происходит инициализация параллельной области. Переменные private изолированы для каждого потока. Каждый поток узнает свой уникальный номер (от 0 до p - 1), а также общее число запущенных нитей local_sum = 0.0d0. С помощью цикла do происходит ручное циклическое распределение итераций между потоками с шагом nt. Средство синхронизации - атомарное сложение локальных результатов нитей в общую сумму. Выполняется как неделимая аппаратная инструкция для защиты от гонки данных.

```
function integrate_reduction(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, s, x
    integer(8) :: i

    h = (b - a) / real(n, 8)
    s = 0.0d0
    !$omp parallel do private(x) reduction(+:s)
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        s = s + log(x) / (1.0d0 + x)
    end do
    !$omp end parallel do

    res = s * h
end function integrate_reduction
```

3.Parallel Do + Reduction (REDUCTION)

```
function integrate_atomic(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, s, p_s, x, val
    integer(8) :: i

    h = (b - a) / real(n, 8)
    s = 0.0d0

    !$omp parallel private(x, val, p_s) shared(s, a, h, n)
    p_s = 0.0d0
    !$omp do

    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        val = log(x) / (1.0d0 + x)
        p_s = p_s + val
    end do
    !$omp end do

    !$omp atomic
    s = s + p_s
    !$omp end parallel

    res = s * h
end function integrate_atomic
```

4.Parallel Do + Atomic (ATOMIC)

В варианте с автоматическим параллельным циклом с редукцией **(REDUCTION)** директива параллелизации цикла автоматического распределения витков do между потоками. Средство синхронизации: опция reduction(+:s) создает локальную копию переменной s для каждого потока и автоматически суммирует их на выходе из цикла.

В варианте с автоматическим параллельным циклом с атомарной синхронизацией **(ATOMIC)** запускается параллельная область с выделением локальной переменной ps под частичную сумму. Нарезка цикла ведется силами OpenMP. Потоки копят результаты независимо в свои ps. Здесь потоки ждут завершения расчетов друг друга. Средство синхронизации - прибавление частичной суммы ps к общему итогу s, защищено на аппаратном уровне процессора для исключения гонки данных.

```
function integrate_critical(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, s, p_s, x, val
    integer(8) :: i

    h = (b - a) / real(n, 8)
    s = 0.0d0

    !$omp parallel private(x, val, p_s) shared(s, a, h, n)
    p_s = 0.0d0
    !$omp do
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        val = log(x) / (1.0d0 + x)
        p_s = p_s + val
    end do
    !$omp end do

    !$omp critical
    s = s + p_s
    !$omp end critical
    !$omp end parallel

    res = s * h
end function integrate_critical
```

5.Parallel Do + Critical (CRITICAL)

```
function integrate_false_sharing(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, x
    integer(8) :: i
    integer :: max_threads, tid
    real(8), allocatable :: partial_sums(:)

    h = (b - a) / real(n, 8)

    max_threads = omp_get_max_threads()
    allocate(partial_sums(0:max_threads-1))
    partial_sums = 0.0d0

    !$omp parallel private(x, tid) shared(partial_sums, a, h, n)
    tid = omp_get_thread_num()

    !$omp do
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        partial_sums(tid) = partial_sums(tid) + log(x) / (1.0d0 + x)
    end do
    !$omp end do
    !$omp end parallel

    res = sum(partial_sums) * h
    deallocate(partial_sums)
end function integrate_false_sharing
```

6.Ложное разделение (FALSE_SHARING)

В варианте с автоматическим циклом с критической секцией **(CRITICAL)** средство синхронизации - критическая секция critical создает программный замок. Потоки выстраиваются в очередь и поочередно прибавляют свои значения к s, исключая возможность одновременной записи.

В варианте сбора данных в общий массив **(FALSE_SHARING)** потоки пишут результаты в независимые ячейки массива по своему индексу tid. Логической гонки данных нет, но близкое расположение данных провоцирует постоянную аппаратную блокировку ядер процессора

6. Результаты проверки правильности выполнения программы

Для проверки корректности работы программ полученный результат сопоставлялся с точным теоретическим (табличным) значением интеграла:

$$I_{\text{теор}} = -\frac{\pi^2}{12} \approx -0{,}8224670334$$

Одиночный последовательный метод (SERIAL) и все пять реализованных параллельных вариантов OpenMP (SPMD, REDUCTION, ATOMIC, CRITICAL, FALSE_SHARING) при числе разбиений отрезка N = 100000000 выдали одинаковое число:

$$I_{\text{расчет}} = -0{,}822467030$$

Практический результат полностью совпал с теоретическим эталоном до седьмого знака после запятой.

Абсолютная погрешность: $\Delta = I_{\text{теор}} - I_{\text{расчет}} = -0{,}8224670334 - (-0{,}822467030) = -0{,}0000000034$

Относительная погрешность: $\delta = \frac{\Delta}{I_{\text{теор}}} = 4{,}1339 \cdot 10^{-9}$

Тот факт, что все параллельные модификации выдали в точности такое же число, что и последовательная версия, доказывает: в коде полностью отсутствует гонка данных, а все примененные средства синхронизации работают математически корректно.

7. Результаты исследования эффективности распараллеливания программы

С помощью языка программирования Python и библиотек Pandas, Matplotlib были построены графики ускорения и эффективности для всех вариантов от числа потоков (см. Рис.1). Результаты работы каждого варианта представлены в таблице 1.

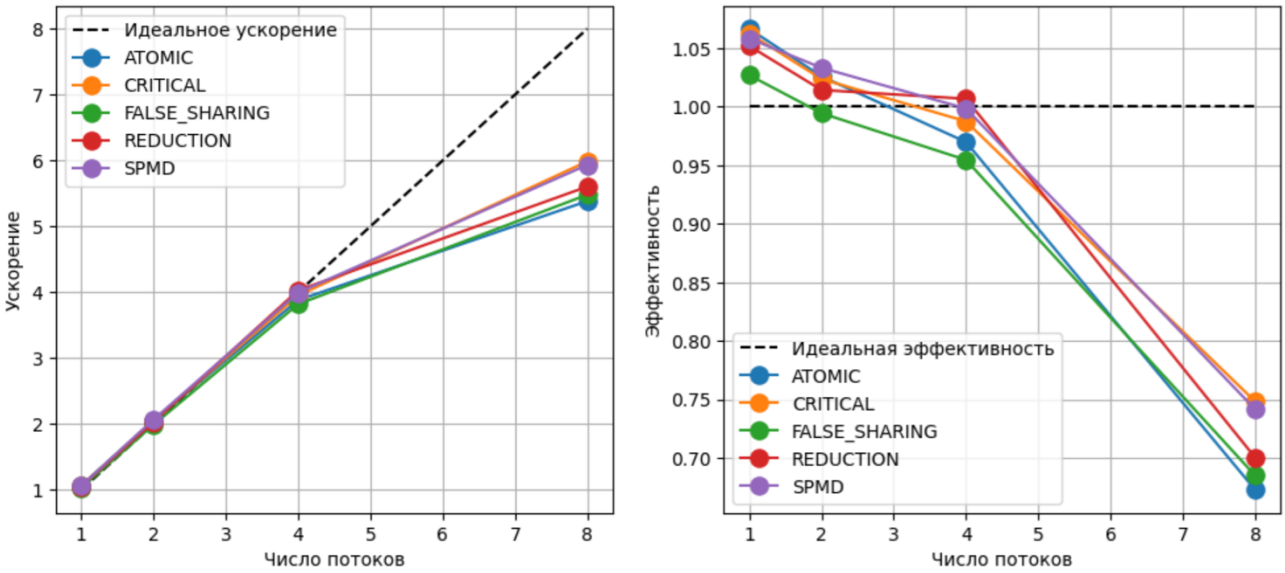


Рисунок 1. Графики ускорения и эффективности от числа потоков

На 2 потоках время упало в два раза (~0.15 сек). Ускорение составило S ≈ 2

На 4 потоках время упало в четыре раза (~0.08 сек). Ускорение составило S ≈ 4

На 2 и 4 потоках все методы дают идеальный результат, потому что задача (100 миллионов логарифмов) делится по ядрам равномерно, а потоки складывают свои суммы всего один раз в самом конце.

На 8 потоках время опустилось до минимума (~0.053–0.060 сек). Максимальное ускорение составило S ≈ 6. Это предсказуемое поведение для процессора M1, так как его вторая половина ядер (4 ядра Efficiency) изначально сделаны более слабыми и энергосберегающими.

Метод с общим массивом не затормозил и отработал на уровне остальных. Это говорит о том, что на современном процессоре M1 очень быстрая память, которая сгладила проблему ложного разделения кэша.

Лучший результат показали методы CRITICAL и SPMD

method	threads	n	integral	time
SPMD	1	100000000	-0.822467030	0.305312000
SPMD	2	100000000	-0.822467030	0.156367000
SPMD	4	100000000	-0.822467030	0.080905000
SPMD	8	100000000	-0.822467030	0.054461000
REDUCTION	1	100000000	-0.822467030	0.307173000
REDUCTION	2	100000000	-0.822467030	0.159278000
REDUCTION	4	100000000	-0.822467030	0.080236000
REDUCTION	8	100000000	-0.822467030	0.057670000
FALSE_SHARING	1	100000000	-0.822467030	0.314635000
FALSE_SHARING	2	100000000	-0.822467030	0.162461000
FALSE_SHARING	4	100000000	-0.822467030	0.084627000
FALSE_SHARING	8	100000000	-0.822467030	0.058964000
CRITICAL	1	100000000	-0.822467030	0.304072000
CRITICAL	2	100000000	-0.822467030	0.157823000
CRITICAL	4	100000000	-0.822467030	0.081806000
CRITICAL	8	100000000	-0.822467030	0.053982000
ATOMIC	1	100000000	-0.822467030	0.302951000
ATOMIC	2	100000000	-0.822467030	0.157479000
ATOMIC	4	100000000	-0.822467030	0.083292000
ATOMIC	8	100000000	-0.822467030	0.060030000
SERIAL	1	100000000	-0.822467030	0.323139000

Таблица 1. Результаты работы всех 6 вариантов при вычислении интеграла

8. Результаты исследования эффективности способов распределения итераций цикла

Была проведена проверка, как разные способы раздачи задач (опция schedule) между 4 потоками влияют на скорость работы метода REDUCTION на 100 млн отрезков:

1. ***Статический режим (STATIC) с разным размером блока (1, 10, 1000):***

```
OMP_NUM_THREADS=4 OMP_SCHEDULE="static,1" ./integral_fortran REDUCTION 100000000

OMP_NUM_THREADS=4 OMP_SCHEDULE="static,10" ./integral_fortran REDUCTION 100000000

OMP_NUM_THREADS=4 OMP_SCHEDULE="static,1000" ./integral_fortran REDUCTION 100000000
```

При размере блока 1 (static,1) время составило 0.092 сек. Оно чуть выше, так как нарезка цикла на порции по 1 прямоугольнику заставляет процессор часто переключаться.

При укрупнении блоков до 10 и 1000 (static,10 и static,1000) время упало до минимума — 0.081 – 0.082 сек. Это лучший результат, потому что нагрузка на всем отрезке одинаковая и ядра загружены равномерно.

2. ***Динамический (DYNAMIC) и направляемый (GUIDED) режимы:***

```
OMP_NUM_THREADS=4 OMP_SCHEDULE="dynamic" ./integral_fortran REDUCTION 100000000

OMP_NUM_THREADS=4 OMP_SCHEDULE="guided" ./integral_fortran REDUCTION 100000000
```

В режиме DYNAMIC время составило 0.092 сек. Оно медленнее, так как потоки постоянно тратят время на обращение к диспетчеру за новыми порциями задач.

В режиме GUIDED (где размер порций уменьшается постепенно) результат получился лучше — 0.083 сек, почти на уровне статики.

Для простых математических интегралов, где сложность расчета одинакова в любой точке отрезка, лучшим выбором является крупноблочная статика (static,10 или static,1000). Динамическое распределение здесь не нужно, так как оно только тратит ресурсы процессора на лишнее управление очередью задач.

Выводы

В ходе лабораторной работы были изучены базовые конструкции OpenMP при распараллеливании численного интегрирования для варианта №5. Включение оптимизации компилятора с флагом -O3сократило время последовательного расчета с 0.45 до 0.33 секунды за счет эффективной сборки математических формул логарифма. Автоматическое распараллеливание силами одного лишь компилятора не сработало из-за циклической зависимости по данным в переменной суммы, что подтвердило необходимость ручного применения OpenMP.

Все разработанные параллельные модификации показали высокую скорость работы и выдали математически точный результат до седьмого знака после запятой, что доказывает отсутствие гонки данных и правильность расчетов. На 2 и 4 потоках было получено идеальное линейное ускорение в 2 и 4 раза соответственно. На максимальных 8 потоках рост производительности замедлился до 6 раз, что связано с разнородной архитектурой процессора M1, у которого только половина ядер являются высокопроизводительными, а остальные четыре — энергоэффективными. Метод со сбором данных в общий массив на процессоре M1 не показал ожидаемых тормозов и отработал на уровне остальных из-за высокой пропускной способности памяти и эффективных встроенных протоколов работы с кэшем, которые сгладили эффект ложного разделения кэш-линий.

Исследование режимов раздачи задач показало, что для простых циклов с равномерной нагрузкой лучше всего подходит крупноблочное статическое распределение итераций с параметрами static,10или static,1000, которое выдало минимальное время 0.081 секунды. Динамический режим DYNAMIC отработал медленнее (0.092 секунды), так как процессор тратил лишнее время на постоянное управление очередью и выдачу мелких порций задач потокам прямо во время вычислений.

Приложение (код программы)

```
module variants_lib
   implicit none

   integer, parameter :: vUNKNOWN = 0
   integer, parameter :: vSERIAL = 1
   integer, parameter :: vSPMD = 2
   integer, parameter :: vREDUCTION = 3
   integer, parameter :: vATOMIC = 4
   integer, parameter :: vCRITICAL = 5
   integer, parameter :: vFALSE_SHARING = 6

   contains

   function var2str(var) result(str)
      integer, intent(in) :: var
      character(len=:), allocatable :: str
      select case (var)
      case (vSERIAL); str = "SERIAL"
      case (vSPMD); str = "SPMD"
      case (vREDUCTION); str = "REDUCTION"
      case (vATOMIC); str = "ATOMIC"
      case (vCRITICAL); str = "CRITICAL"
      case (vFALSE_SHARING); str = "FALSE_SHARING"
      case default; str = "UNKNOWN"
      end select
   end function var2str

   function str2var(str) result(var)
      character(len=*), intent(in) :: str
      integer :: var
      select case (str)
      case ("SERIAL"); var = vSERIAL
      case ("SPMD"); var = vSPMD
      case ("REDUCTION"); var = vREDUCTION
      case ("ATOMIC"); var = vATOMIC
      case ("CRITICAL"); var = vCRITICAL
      case ("FALSE_SHARING"); var = vFALSE_SHARING
      case default; var = vUNKNOWN
      end select
   end function str2var

end module variants_lib

program main
   use variants_lib
   use omp_lib, only: omp_get_wtime, omp_get_max_threads

   implicit none

   integer, parameter :: Action_RUN = 0, Action_ERROR = 1
   real(8), parameter :: a = 0.0d0, b = 1.0d0

   type :: Options
```

```
    integer :: variant = vUNKNOWN
    integer(8) :: ndiv = 0
end type Options

type ParseResult
    integer :: action = Action_RUN
    type(Options) :: options
    character(len=:), allocatable :: error
    character(len=:), allocatable :: argument
end type ParseResult

type(ParseResult) :: parsed
character(len=256) :: prog
character(len=32) :: I_str, Time_str
integer(8) :: n
real(8) :: I, exec_time

interface
    function integrate_sequential(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
    function integrate_spmd(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
    function integrate_reduction(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
    function integrate_atomic(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
    function integrate_critical(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
    function integrate_false_sharing(a, b, n) result(res)
        real(8), intent(in) :: a, b
        integer(8), intent(in) :: n
        real(8) :: res
    end function
end interface

call get_command_argument(0, prog
parsed = parse_input_arguments()

if (parsed%action == Action_ERROR) then
    call report_error(trim(prog), parsed%error, parsed%argument)
    stop 1
end if
```

```
n = parsed%options%ndiv
I = 0.d0

exec_time = omp_get_wtime()
select case (parsed%options%variant)

case (vSERIAL); I = integrate_sequential(a, b, n)
case (vSPMD); I = integrate_spmd(a, b, n)
case (vREDUCTION); I = integrate_reduction(a, b, n)
case (vATOMIC); I = integrate_atomic(a, b, n)
case (vCRITICAL); I = integrate_critical(a, b, n)
case (vFALSE_SHARING); I = integrate_false_sharing(a, b, n)
case default; continue
end select
exec_time = omp_get_wtime() - exec_time

write(I_str, '(F20.9)') I
write(Time_str, '(F20.9)') exec_time
write (*, '(A,",",I0,",",I0,",",A,",",A)') &
   var2str(parsed%options%variant), omp_get_max_threads(),&
   n, trim(adjustl(I_str)), trim(adjustl(Time_str))

contains

subroutine parse_positive_int(token, val, ok)
   character(len=*), intent(in) :: token
   integer(8), intent(inout) :: val
   logical, intent(inout) :: ok
   integer :: i, digit
   ok = .false.
   val = int(0, 8)
   if (len(token) == 0) return
   do i = 1, len_trim(token)
      if (token(i:i) < '0' .or. token(i:i) > '9') return;
      digit = iachar(token(i:i)) - iachar('0')
      val = val * int(10, 8) + int(digit, 8
   end do
   ok = (val > int(0, 8))
end subroutine parse_positive_int

subroutine set_failure(res, message, argument)
   type(ParseResult), intent(inout) :: res
   character(len=*), intent(in) :: message, argument

   res%action = Action_Error
   res%error = message
   res%argument = argument
end subroutine set_failure

function parse_input_arguments() result(result_)
   type(ParseResult) :: result_
   integer :: i, argc, positional_count, arg_len
   character(len=:), allocatable :: token
   logical :: ok

   result_%action = Action_Run
   result_%error = ""
```

```
        result_%argument = ""
        argc = command_argument_count()

        positional_count = 0
        do i = 1, argc
           if (allocated(token)) deallocate(token)
           call get_command_argument(i, length=arg_len)
           allocate(character(len=arg_len) :: token)
           call get_command_argument(i, value=token)

           if (positional_count == 0) then
              positional_count = positional_count + 1
              result_%options%variant = str2var(token)
              if (result_%options%variant == vUNKNOWN) then
                 call set_failure(result_, "Invalid directive name", token)
                 return
              end if
              cycle

           else if (positional_count == 1 .and. len(token) > 0) then
              positional_count = positional_count + 1
              call parse_positive_int(token, result_%options%ndiv, ok)
              if (.not. ok) then
                 call set_failure(result_, "Invalid number of segment division", token)
                 return
              end if
              cycle
           end if
           call set_failure(result_, "Unexpacted argument", token)
           return
        end do

        if (positional_count < 2) call set_failure(result_, "Missing required arguments. Expected: NAME N",
""")
     end function parse_input_arguments

     subroutine report_error(prog_name, message, argument)
        character(len=*), intent(in) :: prog_name, message, argument
        write (*, '(2A)', advance='no') "Error: ", message
        if (len(argument) > 0) write (*, '(3A)', advance='no') " -> '", argument, "'"
        write (*, '(A)') new_line('a')

        write (*, '(A)') "Usage: " // prog_name // " <DIRECTIVE NAME> <NUMBER OF DIVISIONS>"
        write (*, '(A)') "Available names:"
        write (*, '(A)') "  SERIAL SPMD REDUCTION ATOMIC CRITICAL FALSE_SHARING"
     end subroutine report_error

end program main

function integrate_sequential(a, b, n) result(res)
     implicit none
     real(8), intent(in) :: a, b
     integer(8), intent(in) :: n
     real(8) :: res, h, s, x
     integer(8) :: i

     h = (b - a) / real(n, 8)
```

```
    s = 0.0d0
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        s = s + log(x) / (1.0d0 + x)
    end do
    res = s * h
end function integrate_sequential

function integrate_spmd(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, total_sum, local_sum, x
    integer(8) :: i
    integer :: tid, nt

    h = (b - a) / real(n, 8)
    total_sum = 0.0d0

    !$omp parallel private(tid, nt, i, x, local_sum) shared(a, h, n, total_sum)

    tid = omp_get_thread_num()
    nt = omp_get_num_threads()
    local_sum = 0.0d0

    do i = tid + 1, n, nt
        x = a + (real(i, 8) - 0.5d0) * h
        local_sum = local_sum + log(x) / (1.0d0 + x)
    end do

    !$omp atomic
    total_sum = total_sum + local_sum
    !$omp end parallel

    res = total_sum * h
end function integrate_spmd

function integrate_reduction(a, b, n) result(res)
    use omp_lib
    implicit none
    real(8), intent(in) :: a, b
    integer(8), intent(in) :: n
    real(8) :: res, h, s, x
    integer(8) :: i

    h = (b - a) / real(n, 8)
    s = 0.0d0
    !$omp parallel do private(x) reduction(+:s)
    do i = 1, n
        x = a + (real(i, 8) - 0.5d0) * h
        s = s + log(x) / (1.0d0 + x)
    end do
    !$omp end parallel do

    res = s * h
end function integrate_reduction
```

```
function integrate_atomic(a, b, n) result(res)
   use omp_lib
   implicit none
   real(8), intent(in) :: a, b
   integer(8), intent(in) :: n
   real(8) :: res, h, s, p_s, x, val
   integer(8) :: i

   h = (b - a) / real(n, 8)
   s = 0.0d0

   !$omp parallel private(x, val, p_s) shared(s, a, h, n)
   p_s = 0.0d0
   !$omp do

   do i = 1, n
      x = a + (real(i, 8) - 0.5d0) * h
      val = log(x) / (1.0d0 + x)
      p_s = p_s + val
   end do
   !$omp end do

   !$omp atomic
   s = s + p_s
   !$omp end parallel

   res = s * h
end function integrate_atomic

function integrate_critical(a, b, n) result(res)
   use omp_lib
   implicit none
   real(8), intent(in) :: a, b
   integer(8), intent(in) :: n
   real(8) :: res, h, s, p_s, x, val
   integer(8) :: i

   h = (b - a) / real(n, 8)
   s = 0.0d0

   !$omp parallel private(x, val, p_s) shared(s, a, h, n)
   p_s = 0.0d0
   !$omp do
   do i = 1, n
      x = a + (real(i, 8) - 0.5d0) * h
      val = log(x) / (1.0d0 + x)
      p_s = p_s + val
   end do
   !$omp end do

   !$omp critical
   s = s + p_s
   !$omp end critical
   !$omp end parallel

   res = s * h
```

```
end function integrate_critical

function integrate_false_sharing(a, b, n) result(res)
   use omp_lib
   implicit none
   real(8), intent(in) :: a, b
   integer(8), intent(in) :: n
   real(8) :: res, h, x
   integer(8) :: i
   integer :: max_threads, tid
   real(8), allocatable :: partial_sums(:)

   h = (b - a) / real(n, 8)

   max_threads = omp_get_max_threads()
   allocate(partial_sums(0:max_threads-1))
   partial_sums = 0.0d0

   !$omp parallel private(x, tid) shared(partial_sums, a, h, n)
   tid = omp_get_thread_num()

   !$omp do
   do i = 1, n
      x = a + (real(i, 8) - 0.5d0) * h
      partial_sums(tid) = partial_sums(tid) + log(x) / (1.0d0 + x)
   end do
   !$omp end do
   !$omp end parallel

   res = sum(partial_sums) * h
   deallocate(partial_sums)
end function integrate_false_sharing
```